

1. Find the First and Last Movie Released Each Year

- Retrieve the first and last movie released in each year. Display the year, first movie title, and last movie title.

```
SELECT strftime ('%Y', release_date) AS year,  
min(title) AS first_movie,  
max(title) AS last_movie  
FROM movies  
GROUP BY year;
```

	year	first_movie	last_movie
1	1916	Intolerance	Intolerance
2	1925	The Big Parade	The Big Parade
3	1927	Metropolis	Metropolis
4	1929	Pandora's Box	The Broadway Melody
5	1930	Hell's Angels	Hell's Angels
6	1932	A Farewell to Arms	A Farewell to Arms

2. Total Revenue Generated Each Year

- Calculate the total revenue generated by movies released in each year. Display the year and total revenue, ordered by revenue in descending order.

```
SELECT strftime ('%Y', release_date) AS year,  
SUM(revenue) AS total_revenue  
FROM movies  
GROUP BY year  
ORDER BY total_revenue DESC;
```

	year	total_revenue
1	2012	24141710246
2	2014	24120490589
3	2013	23411493295
4	2015	22775024221
5	2009	21072651506
6	2011	20516921160

3. Movies Released in the First Half of the Year

- Find all movies that were released between January and June. Display the movie title, release date, and director name.

```
SELECT title,  
release_date,  
directors.name AS director_name  
FROM movies  
JOIN directors  
ON movies.director_id = directors.id  
WHERE CAST(strftime("%m", release_date) AS INTEGER) BETWEEN 1 AND 6;
```

	title	release_date	director_name
1	Pirates of the Caribbean: At World's End	2007-05-19	Gore Verbinski
2	John Carter	2012-03-07	Andrew Stanton
3	Spider-Man 3	2007-05-01	Sam Raimi
4	Avengers: Age of Ultron	2015-04-22	Joss Whedon
5	Batman v Superman: Dawn of Justice	2016-03-23	Zack Snyder
6	Superman Returns	2006-06-28	Bryan Singer

4. Number of Movies Released on Weekends

- Count the total number of movies released on a weekend (Saturday or Sunday). Use SQLite's strftime('%w') to determine the day of the week (0 = Sunday, 6 = Saturday).

```
SELECT count(*) AS weekend_movies  
FROM movies  
WHERE strftime('%w', release_date) IN ('0', '6');
```

	weekend_movies
1	451

5. Categorize Directors Based on Total Revenue

- Classify directors based on the total revenue of their movies:
 - High Revenue if total revenue > \$1 billion
 - Moderate Revenue if total revenue is between \$500 million and \$1 billion
 - Low Revenue if total revenue < \$500 million

```
SELECT directors.name,  
sum(movies.revenue) AS total_revenue,  
CASE
```

```

        WHEN sum(movies.revenue) > 1000000000 THEN 'High Revenue'
        WHEN sum(movies.revenue) >= 500000000 THEN 'Moderate Revenue'
        ELSE 'Low Revenue'
END AS revenue_category
FROM movies
JOIN directors
ON movies.director_id = directors.id
GROUP BY directors.name;

```

	name	total_revenue	revenue_category	
1	Aaron Hann	0	Low Revenue	
2	Aaron Schneider	0	Low Revenue	
3	Abel Ferrara	1227324	Low Revenue	
4	Adam Brooks	55447968	Low Revenue	
5	Adam Carolla	0	Low Revenue	
6	Adam Goldberg	0	Low Revenue	

6. Movies Based on Popularity and Revenue

- Classify movies into the following categories based on popularity and revenue:
 - Blockbuster if popularity > 100 and revenue > \$500 million
 - Hit if popularity > 50 and revenue between \$100 million and \$500 million
 - Average for all other movies

```

SELECT title,
popularity,
revenue,
CASE
    WHEN popularity > 100 AND revenue > 500000000 THEN 'Blockbuster'
    WHEN popularity > 50 AND revenue BETWEEN 100000000 AND 500000000 THEN
'Hit'
    ELSE 'Average'
END AS popularity_category
FROM movies;

```

	title	popularity	revenue	pop
1	Avatar	150	2787965087	Block
2	Pirates of the Caribbean: At World's End	139	961000000	Block
3	Spectre	107	880674609	Block
4	The Dark Knight Rises	112	1084939099	Block
5	John Carter	43	284139100	Aver
6	Spider-Man 3	115	890871626	Block

7. Directors with Critically Acclaimed and Poorly Rated Movies

- List directors and count how many of their movies fall into these categories:

- Critically Acclaimed if vote_average >= 8
- Poorly Rated if vote_average < 5

```

SELECT directors.name,
sum(CASE
    WHEN movies.vote_average >= 8 THEN 1
    ELSE 0
END) AS critically_acclaimed,
sum(CASE
    WHEN movies.vote_average < 5 THEN 1
    ELSE 0
END) AS poorlyRated
FROM movies
JOIN directors
ON movies.director_id = directors.id
GROUP BY directors.name;

```

	name	critically_acclaimed	poorlyRated	
1	Aaron Hann	0	0	
2	Aaron Schneider	0	0	
3	Abel Ferrara	0	0	
4	Adam Brooks	0	0	
5	Adam Carolla	0	0	
6	Adam Goldberg	0	0	

No AI was used.